INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences

Maidan Garhi, New Delhi – 110 068

Project Report

Course Code: **BCSP-064** | BCA – 6th Semester

Hotel Room Booking Website with Assistant Chatbot

Submitted in partial fulfilment for the award of the degree of

Bachelor of Computer Applications (BCA)

Submitted By

Name: **Nikhil P N**

Enrolment No: **2350526613**

Programme: BCA

Regional Centre: **IGNOU Cochin Regional Centre**

Study Centre Code: _____

Under the Guidance of

Guide Name: _____

Designation: _____

Qualification: _____

Address: _____

Academic Year: **2025 – 2026**

Certificate of Originality

This is to certify that the project report entitled **"Hotel Room Booking Website with Assistant Chatbot"** submitted to Indira Gandhi National Open University (IGNOU) in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)** is an original piece of work carried out by **Mr. Nikhil P N**, Enrolment Number **2350526613**, under the guidance of **Mrs. Anjali.v**.

The matter embodied in this project report is a genuine work done by the student and has not been submitted in part or in full to this University or any other University or Institute for the fulfilment of any course requirement.

Signature of the Student

Name: Nikhil P N

Enrolment No: 2350526613

Date: _____

Signature of the Guide

Name: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to **Indira Gandhi National Open University (IGNOU)** for providing me the opportunity to undertake this project as part of the BCA programme. I am deeply indebted to my project guide, **Mrs. Anjali.v**, whose valuable guidance, constant encouragement, and constructive suggestions throughout the duration of this project enabled me to complete it successfully.

I extend my thanks to the IGNOU Cochin Regional Centre and my Study Centre for their administrative support. I also acknowledge the open-source community and the developers of Django, Python, PostgreSQL, Razorpay, and Bootstrap for their excellent tools and documentation that made this project possible.

Last but not least, I express my deepest gratitude to my family and friends for their constant moral support and encouragement throughout this project work.

Nikhil P N

Enrolment No: 2350526613

BCA – 6^{th} Semester

Table of Contents

1. Introduction

   1.1 Overview · 1.2 Problem Statement · 1.3 Solution · 1.4 Existing System · 1.5 Objectives

2. Tools and Environment Used

   2.1 Languages & Libraries · 2.2 Tools

3. Requirement Specification

   3.1 Overall Description · 3.2 Specific Requirements · 3.3 Use Case Diagram

4. Design Document

   4.1 Modularization · 4.2 Data Integrity · 4.3 Procedural Design · 4.4 UI Design · 4.5 Database Design

5. Program Code
6. Testing

   6.1 Unit Testing · 6.2 Integration Testing · 6.3 Code Improvement

7. Input and Output Screens
8. Implementation Plan
9. Limitations of the Project
10. Future Application of the Project
11. Conclusion
12. Bibliography

Abstract

The **Hotel Room Booking Website with Assistant Chatbot** is a full-stack web application developed using Python, Django 6.0, and PostgreSQL. The system automates hotel operations including room-type management, room inventory, customer registration, online booking with date-based availability checking, Razorpay payment processing, staff administration, contact handling, email notifications, PDF report generation, and an intelligent assistant chatbot.

The project follows the SDLC encompassing Requirements Analysis, System Design, Coding, Testing, and Documentation. It consists of eight functional modules spanning the administrator backend and customer-facing website, with RESTful API endpoints for the chatbot and advanced search. The administrator can manage room types, rooms, staff, bookings, payments, and chatbot responses from a secure backend. Customers can register, browse rooms, search with filters, book with overlap detection, pay via Razorpay, interact with the chatbot, and reset passwords via OTP.

Keywords: Hotel Room Booking, Django, Python, PostgreSQL, Chatbot, Razorpay, AJAX, Bootstrap, SDLC

1. Introduction

1.1 Overview

The hospitality industry serves millions of travellers daily and faces challenges in managing room inventory, bookings, payments, and guest communication. In many hotels, these are still done via paper registers, phone reservations, and spreadsheets, leading to duplicate bookings, inaccurate availability, and poor reporting. Customers increasingly expect online search, booking, and payment capabilities. This project addresses these challenges by building a web-based Hotel Room Booking System using Django 6.0 and PostgreSQL, serving both customers and administrators.

1.2 Problem Statement

1. **Manual Processes:** Paper registers and phone calls are error-prone and slow.
2. **Double Booking:** Without automated checking, rooms may be double-booked.
3. **Limited Reach:** Hotels without online booking lose customers who prefer digital channels.
4. **No 24/7 Support:** Guest enquiries can only be answered during front-desk hours.
5. **Inefficient Reporting:** Generating occupancy/revenue reports is difficult manually.

1.3 Solution to the Problem

The system provides a centralized platform with: a normalized PostgreSQL

database with referential integrity; real-time date-overlap detection preventing double booking; online booking and Razorpay payment gateway; an assistant chatbot for 24/7 support; automated email notifications; on-screen and PDF reporting; and session-based authentication with CSRF protection.

1.4 Existing System

Existing systems rely on paper registers, phone reservations, and spreadsheets. Limitations include no real-time availability, no online access, no automated communication, difficult reporting, and security concerns. The proposed system replaces all of these with a unified digital platform.

1.5 Objectives

1. Design a web-based booking system automating room, booking, and staff management using Django.
2. Provide responsive UI for browsing, searching (by name, type, price, dates), and viewing rooms.
3. Implement date-overlap detection preventing double booking.
4. Integrate Razorpay for secure online payments.
5. Build a rule-based chatbot for room/staff/booking queries.
6. Implement AJAX-based advanced search without page reload.
7. Send email notifications for bookings and OTP password reset.
8. Generate PDF reports via xhtml2pdf.
9. Maintain security via authentication, CSRF, and input validation.
10. Follow SDLC practices throughout development.

2. Tools and Environment Used

2.1 Programming Languages and Libraries

Category	Technology	Purpose
Backend	Python 3.10+, Django 6.0	Web framework (MVT architecture)
Database	PostgreSQL 16	Relational database management
Frontend	HTML5, CSS3, Bootstrap 5.3	Responsive UI
Frontend	JavaScript ES6, AJAX (Fetch API)	Dynamic interactions, async operations
Payment	Razorpay Python SDK	Online payment gateway
PDF	xhtml2pdf 0.2+, reportlab	PDF report generation
Email	Django SMTP (Gmail)	Email notifications and OTP
Config	python-decouple	Environment variable management
Images	Pillow	ImageField processing

2.2 Tools

Tool	Purpose
VS Code	Code editor with debugging and Git integration
Git	Version control
Chrome DevTools	Debugging and testing
Python venv	Virtual environment management
LibreOffice	Document editing and PDF conversion

3. Requirement Specification

3.1 Overall Description

3.1.1 Product Perspective

The system is a web-based client-server application. The client is a web browser rendering Bootstrap 5.3 pages with JavaScript. The server runs Django 6.0 processing HTTP requests with business logic and PostgreSQL database via ORM. Users include Customers (guests) and Administrators (hotel staff). External integrations include Razorpay for payments and Gmail SMTP for emails.

3.1.2 Product Functions

Customer: Registration, login/logout, password reset via OTP, browse room categories, view rooms, advanced search (AJAX), book with overlap detection, cart with tax calculation, Razorpay payment, contact form, chatbot interaction, rate responses, email notifications.

Administrator: Login, CRUD room types/rooms/staff, view/delete contacts, view bookings/payments, manage chatbot responses via Django Admin.

3.1.3 User Characteristics

Customers: General public needing intuitive, responsive interface. **Administrators:** Hotel staff requiring secure form-based CRUD operations and tabular data views.

3.1.4 Constraints

Requires Python 3.10+, Django 6.0, PostgreSQL 16. Single-hotel design. Rule-based chatbot (no NLP). Plain-text passwords in custom Registerdb (known limitation).

No mobile app.

3.2 Specific Requirements

3.2.1 Functional Requirements

ID	Module	Requirement
FR01	User	Registration with username, email, password
FR02	User	Login/logout with session management
FR03	User	Password reset via email OTP
FR04	Room	Admin CRUD room types with image
FR05	Room	Admin CRUD rooms (FK to type)
FR06	Staff	Admin CRUD staff with social links
FR07	Search	Browse room categories and filtered rooms
FR08	Search	AJAX advanced search (name, type, price, dates)
FR09	Booking	Book with date-overlap detection
FR10	Booking	Cart with subtotal/tax/total
FR11	Payment	Razorpay order creation and processing
FR12	Chatbot	Query classification and response
FR13	Chatbot	Response rating (1-5)
FR14	Notify	Booking confirmation email
FR15	Notify	OTP email for password reset
FR16	Report	PDF booking report
FR17	Admin	View bookings and payments

3.2.2 Non-Functional Requirements

Attribute	Requirement
Performance	Pages load within 2-3 sec; AJAX responses within 1 sec
Security	CSRF tokens on all forms; ORM parameterised queries; template auto-escaping
Usability	Responsive at 1920px, 768px, 375px; consistent navigation

Reliability	Input validation on all forms; date-overlap prevents conflicts
Maintainability	Modular Backend/webapp Django apps; separation of concerns

3.2.3 Performance Requirements

Response within 2 seconds for pages, 1 second for AJAX. Support 50 concurrent users. Optimized ORM queries with select_related().

3.2.4 External Interface Requirements

UI: Chrome 120+, Edge 120+, Firefox 120+ at 375px-1920px widths. **Payment:** Razorpay JavaScript SDK popup + Python SDK. **Email:** Gmail SMTP. **API:** JSON REST endpoints for chatbot and search.

3.2.5 System Evolution

Modular architecture allows adding multi-hotel support, dynamic pricing, mobile APIs, and analytics dashboards as new Django apps.

3.3 Use Case Diagram

Actors: Customer and Administrator.

Customer use cases: Register, Login, Browse Rooms, Advanced Search, Book Room, Make Payment, Submit Contact, Chat with Bot, Reset Password, View Cart, Delete Booking.

Administrator use cases: Login, Manage Room Types, Manage Rooms, Manage Staff, View Bookings, View Payments, View Contacts, Manage Chatbot.

4. Design Document

4.1 Modularization Details

4.1.1 Front-End Modules

Module	Pages	Description
Home & Navigation	Home, About, Services, Rooms, Contact, Team	Responsive pages with dynamic DB content, fixed navbar
User Auth	Sign-up, Sign-in, Password Reset (Email→OTP→New)	Registration, login/session, password recovery flow
Room Search & Booking	Categories, Filtered Rooms, Advanced Search, Booking Form, Cart	Browse, AJAX search, book with overlap detection, cart with tax
Payment	Payment Page	Razorpay integration for online payment
Chatbot	Floating chat widget (all pages)	Query classification and contextual responses
Contact	Contact Form	Customer enquiries stored in DB

4.1.2 Back-End Modules

Module	Functions
Admin Dashboard	Navigation to all admin sections
Room Type Management	CRUD room types with image upload
Room Management	CRUD rooms with FK to type
Staff Management	CRUD staff with social links
Contact Management	View/delete contact messages
Booking Management	View all booking records
Payment Management	View payment records
Chatbot & Notifications	Manage responses, conversations via Django Admin

4.2 Data Integrity

4.2.1 Product Data

Room types and rooms are linked via foreign key (on_delete=CASCADE). Deleting a room type cascades to its rooms.

4.2.2 User Data

Registerdb stores user credentials with unique PK. Bookings reference users via FK (on_delete=SET_NULL) preserving history.

4.2.3 Order Data

Booking records link to rooms and customers via FKs. Payment records (Totaldb) reference bookings (on_delete=CASCADE). Date-overlap detection prevents double booking.

4.3 Procedural Design

4.3.1 User Registration and Login

Registration: User fills form → validates → creates Registerdb → redirects to sign-in.

Login: User enters credentials → checks Registerdb → creates session USERNAME → redirects to home.

Password Reset: Email → verify exists → generate 5-digit OTP → store in session → send email → user enters OTP → compare (strip whitespace) → if match, show new password form → update password → send notification.

4.3.2 Room Browsing and Selection

Home page shows room type cards. Clicking a type filters rooms. Advanced search uses AJAX: JavaScript captures filter values → sends GET to search_rooms

API → Django builds filtered queryset, checks availability → returns JSON → JavaScript renders room cards.

4.3.3 Cart

Cart displays all user bookings with subtotal, tax (10% if >₹5000, else flat), and total. Delete button removes individual bookings.

4.3.4 Payment Processing

Customer enters mobile → retrieves Totaldb → amount × 100 (paise) → Razorpay order created → checkout popup → customer pays via card/UPI/net banking.

4.3.5 Order Confirmation

After booking, confirmation email sent via Django SMTP. Booking visible in admin panel.

4.3.6 Administrative Functions

Admin performs CRUD via form-based pages with tabular displays, edit/delete actions, and sidebar navigation.

4.4 User Interface Design

4.4.1 User-Centric Design

Card-based layout using Bootstrap 5.3. Fixed-top navbar with hotel logo, menu items, and auth links. Fully responsive at desktop/tablet/mobile. Floating chatbot icon always accessible.

4.4.2 Key UI Elements

Hero Section: Full-width background with CTA. **Room Cards:** Images, prices, descriptions, "Book Now" button. **Search Panel:** Side filters + AJAX results. **Chatbot:** Floating icon → chat window with suggestions. **Admin Tables:** Striped

rows with edit/delete actions.

4.4.3 Technology and Principles

HTML5 for structure, Bootstrap 5.3 for styling and responsiveness, JavaScript for dynamic interactions (Fetch API). Django templates render server-side with context data.

4.5 Database Design

4.5.1 Entities

Table	Key Fields
roomtypedb	ROOMTYPE, ROOMTYPEIMAGE, DESCRIPTION
roomnamedb	ROOMTYPE (FK), ROOMNAME, ROOMIMAGE, ROOMPRICE, ROOMDESCRIPTION
Registerdb	USERNAME, PASSWORD, EMAIL
bookingdb	CUSTOMER (FK), SELECTROOM (FK), CHECKIN, CHECKOUT, TOTALPRICE
Totaldb	BOOKING (FK), CUSTOMERNAME, MOBILE, TOTALPRICE
staffdb	STAFFNAME, STAFFDESIGNATION, STAFFIMAGE
customercontactdb	CONTACTNAME, CONTACTEMAIL, CONTACTMESSAGE
ChatbotConversation	username, user_message, bot_response, query_type, rating
ChatbotResponse	keywords, response_text, category, priority
Notification	username, notification_type, title, message, is_read

4.5.2 Relationships

roomtypedb 1:N roomnamedb | roomnamedb 1:N bookingdb | Registerdb 1:N bookingdb | bookingdb 1:N Totaldb

4.5.3 Normalization

Database is in 3NF: 1NF (atomic values, PKs), 2NF (no partial dependencies on single-column PKs), 3NF (no transitive dependencies - FKs reference related tables instead of duplicating data).

4.5.4 Referential Integrity

roomnamedb.ROOMTYPE → roomtypedb (CASCADE) | bookingdb.CUSTOMER → Registerdb (SET_NULL) | bookingdb.SELECTROOM → roomnamedb (CASCADE) | Totaldb.BOOKING → bookingdb (CASCADE)

5. Program Code

This section presents the source code of the most important modules. The complete source code is available in the submitted project files.

5.1 Backend Models – Backend/models.py

```
from django.db import models


class roomtypedb(models.Model):
    ROOMTYPE = models.CharField(max_length=100, null=True, blank=True)
    ROOMTYPEIMAGE = models.ImageField(upload_to="roomtypeimages", null=True, blank=True)
    DESCRIPTION = models.CharField(max_length=100, null=True, blank=True)
    def __str__(self): return self.ROOMTYPE or "No Type"


class roomnamedb(models.Model):
    ROOMTYPE = models.ForeignKey(roomtypedb, on_delete=models.CASCADE, null=True, blank=True)
    ROOMNAME = models.CharField(max_length=100, null=True, blank=True)
    ROOMIMAGE = models.ImageField(upload_to="roomimage", null=True, blank=True)
    ROOMPRICE = models.IntegerField(null=True, blank=True)
    ROOMDESCRIPTION = models.CharField(max_length=100, null=True, blank=True)
    def __str__(self): return self.ROOMNAME or "No Room Name"


class staffdb(models.Model):
    STAFFNAME = models.CharField(max_length=100, null=True, blank=True)
    STAFFDESIGNATION = models.CharField(max_length=100, null=True, blank=True)
    STAFFACEBOOK = models.CharField(max_length=100, null=True, blank=True)
    STAFFINSTA = models.CharField(max_length=100, null=True, blank=True)
    STAFFIMAGE = models.ImageField(upload_to="staffimage", null=True, blank=True)
    def __str__(self): return self.STAFFNAME or "No Name"
```

5.2 Webapp Models – webapp/models.py

```
from django.db import models
from Backend.models import roomnamedb
```

```
class customercontactdb(models.Model):
    CONTACTNAME = models.CharField(max_length=100, null=True, blank=True)
    CONTACTNUMBER = models.CharField(max_length=15, null=True, blank=True)
    CONTACTEMAIL = models.EmailField(max_length=100, null=True, blank=True)
    CONTACTSUBJECT = models.CharField(max_length=200, null=True, blank=True)
    CONTACTMESSAGE = models.TextField(null=True, blank=True)
    def __str__(self): return self.CONTACTNAME or "No Name"

class Registerdb(models.Model):
    USERNAME = models.CharField(max_length=100, null=True, blank=True)
    PASSWORD = models.CharField(max_length=100, null=True, blank=True)
    CONFIRMPASSWORD = models.CharField(max_length=100, null=True, blank=True)
    EMAIL = models.EmailField(max_length=100, null=True, blank=True)
    def __str__(self): return self.USERNAME or "No Username"

class bookingdb(models.Model):
    CUSTOMER = models.ForeignKey(Registerdb, on_delete=models.SET_NULL, null=True, blank=True)
    CUSTOMERNAME = models.CharField(max_length=100, null=True, blank=True)
    CONTACTEMAIL = models.EmailField(max_length=100, null=True, blank=True)
    CHECKIN = models.DateField(null=True, blank=True)
    CHECKOUT = models.DateField(null=True, blank=True)
    TOTALADULTS = models.IntegerField(null=True, blank=True)
    TOTALCHILDS = models.IntegerField(null=True, blank=True)
    SELECTROOM = models.ForeignKey(roomnamedb, on_delete=models.CASCADE, null=True, blank=True)
    SPECIALREQUEST = models.TextField(null=True, blank=True)
    TOTALPRICE = models.IntegerField(null=True, blank=True)

class Totaldb(models.Model):
    BOOKING = models.ForeignKey(bookingdb, on_delete=models.CASCADE, null=True, blank=True)
    CUSTOMERNAME = models.CharField(max_length=100, null=True, blank=True)
    MOBILE = models.CharField(max_length=15, null=True, blank=True)
    TOTALPRICE = models.IntegerField(null=True, blank=True)
```

5.3 Booking with Overlap Detection

```
def save_booking_page(request, CONTACTEMAIL=None):
    if request.method == "POST":
        na = request.POST.get('bname'); em = request.POST.get('bemail')
        chn = request.POST.get('bcheckin'); cho = request.POST.get('bcheckout')
        ta = request.POST.get('btotaladults'); tc = request.POST.get('btotalchilds')
```

```
        sr_id = request.POST.get('bselectroom')
        sre = request.POST.get('bspecialrequest'); tp = request.POST.get('btotalprice')
        room_obj = roomnamedb.objects.get(id=sr_id)
        overlapping = bookingdb.objects.filter(SELECTROOM=room_obj
            ).filter(CHECKIN__lte=cho, CHECKOUT__gte=chn)
        if overlapping.exists():
            messages.error(request, "Room already booked for these dates.")
            return redirect('save_room_page')
        customer_obj = None
        if 'USERNAME' in request.session:
            try: customer_obj = Registerdb.objects.get(USERNAME=request.session['USERNAME'])
            except: pass
        obj = bookingdb(CUSTOMER=customer_obj, CUSTOMERNAME=na, CONTACTEMAIL=em,
            CHECKIN=chn, CHECKOUT=cho, TOTALADULTS=int(ta) if ta else None,
            TOTALCHILDS=int(tc) if tc else None, SELECTROOM=room_obj,
            SPECIALREQUEST=sre, TOTALPRICE=int(tp) if tp else None)
        obj.save()
        messages.success(request, "Room booked successfully.")
        send_mail("Congratulations - Room Booked", "Booking successful! Thank you.",
                  EMAIL_HOST_USER, [obj.CONTACTEMAIL], fail_silently=True)
        return redirect('save_room_page')
```

5.4 Advanced Search API

```
def search_rooms(request):
    name = request.GET.get('name',''); room_type = request.GET.get('room_type','')
    min_price = request.GET.get('min_price',''); max_price = request.GET.get('max_price','')
    check_in = request.GET.get('check_in',''); check_out = request.GET.get('check_out','')
    rooms = roomnamedb.objects.all()
    if name: rooms = rooms.filter(ROOMNAME__icontains=name)
    if room_type: rooms = rooms.filter(ROOMTYPE__ROOMTYPE__icontains=room_type)
    if min_price: rooms = rooms.filter(ROOMPRICE__gte=min_price)
    if max_price: rooms = rooms.filter(ROOMPRICE__lte=max_price)
    if check_in and check_out:
        booked = bookingdb.objects.filter(CHECKIN__lte=check_out, CHECKOUT__gte=check_in
            ).values_list('SELECTROOM', flat=True)
        rooms = rooms.exclude(id__in=booked)
    data = [{'id':r.id,'name':r.ROOMNAME,'type':str(r.ROOMTYPE),'price':r.ROOMPRICE,
             'image':r.ROOMIMAGE.url if r.ROOMIMAGE else '','description':r.ROOMDESCRIPTION}
            for r in rooms]
```

```
    return JsonResponse(data, safe=False)
```

5.5 Chatbot Engine

```
def generate_chatbot_response(user_message, username):

    room_kw = ['room','price','availability','luxury','deluxe','suite']

    staff_kw = ['staff','team','manager','receptionist']

    booking_kw = ['booking','reserve','book','check-in','check-out']

    if any(k in user_message for k in room_kw): return handle_room_query(user_message, username)

    elif any(k in user_message for k in staff_kw): return handle_staff_query(user_message,
username)

    elif any(k in user_message for k in booking_kw): return handle_booking_query(user_message,
username)

    else: return handle_general_query(user_message, username)



def handle_room_query(user_message, username):

    response = {'query_type':'room','suggestions':['Show budget rooms','Show luxury rooms']}

    if 'price' in user_message:

        response['response'] = "We offer rooms at various price points. Luxury suites from ₹8,000,
deluxe from ₹5,000, standard from ₹2,500."

    else:

        count = roomnamedb.objects.count()

        response['response'] = f"We have {count} rooms across luxury, deluxe, and standard
categories!"

    return response



def handle_staff_query(user_message, username):

    count = staffdb.objects.count()

    return {'query_type':'staff','response':f"Our hotel has {count} team members including
receptionists, housekeepers, and chefs!",

            'suggestions':['View staff','Contact reception']}



def handle_booking_query(user_message, username):

    return {'query_type':'booking','response':'Browse rooms, select dates, confirm booking, pay via
Razorpay. Confirmation email sent!',

            'suggestions':['Browse rooms','Check my bookings','Cancellation policy']}



def handle_general_query(user_message, username):

    if any(g in user_message for g in ['hello','hi','hey']):

        return {'query_type':'general','response':"Hello! 👋 Welcome! I'm your assistant. Ask me
about rooms, staff, or bookings!",

                'suggestions':['Show rooms','Room prices','Staff details','Booking help']}

    return {'query_type':'general','response':"I'm here to help with rooms, staff, bookings, and
general info!",
```

```
            'suggestions':['Room availability','Hotel services','Check-in time']}
```

5.6 Payment Integration

```
def payment_page(request):
    customer = Totaldb.objects.order_by('-id').first()
    amount = int(customer.TOTALPRICE * 100)  # Razorpay expects paise
    if request.method == "POST":
        client = razorpay.Client(auth=('rzp_test_klfWwvjaFXjXmC', 's9P7dwOwYckK352FfRJOXIRV'))
        payment = client.order.create({'amount': amount, 'currency': 'INR', 'payment_capture':
'1'})
    return render(request, "payment.html", {'customer': customer, 'payy_str': str(amount)})


def save_payment_input(request):
    if request.method == "POST":
        name = request.POST.get('customer_name'); mobile = request.POST.get('mobile')
        total = request.POST.get('total_price')
        last_booking = bookingdb.objects.filter(CUSTOMERNAME=name).order_by('-id').first()
        obj = Totaldb(BOOKING=last_booking, CUSTOMERNAME=name, MOBILE=mobile,
                      TOTALPRICE=int(total) if total else None)
        obj.save(); return redirect('payment_page')
```

5.7 OTP-Based Password Reset

```
def reset_password_email_verification(request):
    if request.method == "POST":
        email = request.POST.get('email')
        if Registerdb.objects.filter(EMAIL=email).exists():
            user = Registerdb.objects.get(EMAIL=email)
            u_otp = random.randint(10000, 55555)
            request.session['otp'] = u_otp
            send_mail("Forgot Password", f"OTP: {u_otp}", EMAIL_HOST_USER, [email],
fail_silently=True)
            return render(request, "16password_reset_otp.html", {'message':"OTP
sent!",'user':user})
        return render(request, "15password_reset_email.html", {'error':"Invalid email"})


def passwordReset_verify_otp(request, user_id):
    user = Registerdb.objects.get(id=user_id)
    u_otp = str(request.POST.get('otp')).strip(); s_otp = str(request.session.get('otp')).strip()
    if u_otp == s_otp:
```

```
        return render(request, "17reset_password.html", {'user':user,'username':user.USERNAME})

    return render(request, "16password_reset_otp.html", {'error':"OTP does not
match!",'user':user})



def reset_password(request, username):

    user = Registerdb.objects.get(USERNAME=username)

    p1 = request.POST.get('password1'); p2 = request.POST.get('password2')

    if p1 == p2:

        Registerdb.objects.filter(USERNAME=username).update(PASSWORD=p1)

        messages.success(request, "Password reset!")

        send_mail("Password Changed", "Your password has been changed.", EMAIL_HOST_USER,
[user.EMAIL], fail_silently=True)

        return redirect("user_login_page")

    return render(request, "17reset_password.html", {'error':"Passwords do not
match!",'username':username})
```

6. Testing

Testing involved four levels: Unit Testing, Integration Testing, System Testing, and User Acceptance Testing.

6.1 Unit Testing

ID	Test Case	Result
UT-01	User registration with valid data	Pass √
UT-02	User login with valid credentials	Pass √
UT-03	User login with wrong password	Pass √
UT-04	Add room type with image	Pass √
UT-05	Add room with FK to type	Pass √
UT-06	Book room – no overlap	Pass √
UT-07	Book room – overlapping dates	Pass √
UT-08	Cart subtotal/tax calculation	Pass √
UT-09	OTP generation (random 5-digit)	Pass √
UT-10	OTP verification – correct	Pass √
UT-11	OTP verification – incorrect	Pass √
UT-12	Password reset – matching passwords	Pass √
UT-13	Chatbot – room query	Pass √
UT-14	Chatbot – staff query	Pass √
UT-15	Chatbot – greeting	Pass √
UT-16	Advanced search – price range	Pass √
UT-17	Advanced search – date availability	Pass √
UT-18	PDF generation	Pass √

6.2 Integration Testing

ID	Modules	Test Case	Result
IT-01	Registration → Login → Booking	User registers, logs in, books – FK links to Registerdb	Pass ✓
IT-02	Booking → Payment	Booking saved → cart → Totaldb with FK → Razorpay order	Pass ✓
IT-03	Room Type → Room → Search	Type created → room added → search returns correct result	Pass ✓
IT-04	Contact Form → Admin View	Customer submits → appears in admin panel	Pass ✓
IT-05	Chatbot → Conversation Storage	User chats → conversation saved → visible in Django Admin	Pass ✓
IT-06	Booking → Email	Booking saved → confirmation email sent	Pass ✓
IT-07	Password Reset → OTP → Update	Email → OTP generated/emailed → verified → password updated	Pass ✓
IT-08	Search with Dates → Overlap	Search excludes rooms booked for those dates	Pass ✓

6.3 Code Improvement

6.3.1 Refactoring

Models separated into dedicated files per Django app. Business logic moved from templates to views. Chatbot logic extracted into chatbot_views.py. Admin views consolidated into reusable patterns.

6.3.2 Optimization

ORM queries use select_related() for FK lookups. AJAX search uses filtered querysets. Chatbot uses efficient keyword matching with early returns. Images served directly from media directory.

6.3.3 Code Reviews

Consistent naming conventions (PascalCase for models, UPPERCASE for fields, snake_case for functions). Error handling for all DB lookup operations. Django best practices followed throughout.

6.3.4 Version Control

Git used throughout with descriptive commit messages after each major feature implementation.

6.4 System Testing

ID	Scenario	Result
ST-01	Complete customer journey: Register → Sign in → Browse → Search → Book → Cart → Pay → Sign out	Pass √
ST-02	Admin management cycle: Login → Add type → Add room → Add staff → View bookings/payments	Pass √
ST-03	Chatbot end-to-end: Room query → Staff query → Booking query → Greeting → Rating	Pass √
ST-04	Password reset: Forgot → Email → OTP → New password → Login with new password	Pass √
ST-05	Responsive – Desktop (1920×1080)	Pass √
ST-06	Responsive – Tablet (768×1024)	Pass √
ST-07	Responsive – Mobile (375×667)	Pass √
ST-08-10	Cross-browser: Chrome, Edge, Firefox	Pass √

6.5 User Acceptance Testing

ID	Role	Task	Feedback	Status
UAT-01	Customer	Register, browse, book, pay	"Smooth process. Easy to find and book a room."	Accepted √
UAT-02	Customer	Use chatbot for room info	"Chatbot answered quickly. Room cards were helpful."	Accepted √

UAT-03	Customer	Reset password via OTP	"Received OTP promptly. Reset without issues."	Accepted ✓
UAT-04	Admin	Manage rooms, staff, view bookings	"Admin panel straightforward. Easy to manage."	Accepted ✓

6.6 Bug Fixes

Bug	Issue	Fix
BUG-01	Double booking allowed	Implemented CHECKIN__lte/CHECKOUT__gte range logic
BUG-02	Image not displaying after edit	Added enctype="multipart/form-data" to edit form
BUG-03	Chatbot empty response	Converted message to lowercase before keyword match
BUG-04	FK assignment error	Used get() to retrieve model object before assignment
BUG-05	OTP comparison failed	Added .strip() to both input and session values
BUG-06	Payment amount incorrect	Multiplied by 100 for Razorpay paise format
BUG-07	Staff page crash (no image)	Added conditional for null ImageField
BUG-08	Search by type returned all	Fixed FK lookup: ROOMTYPE__ROOMTYPE__icontains

7. Input and Output Screens

The following screenshots show the key pages of the Hotel Room Booking Website.

7.1 Home Page

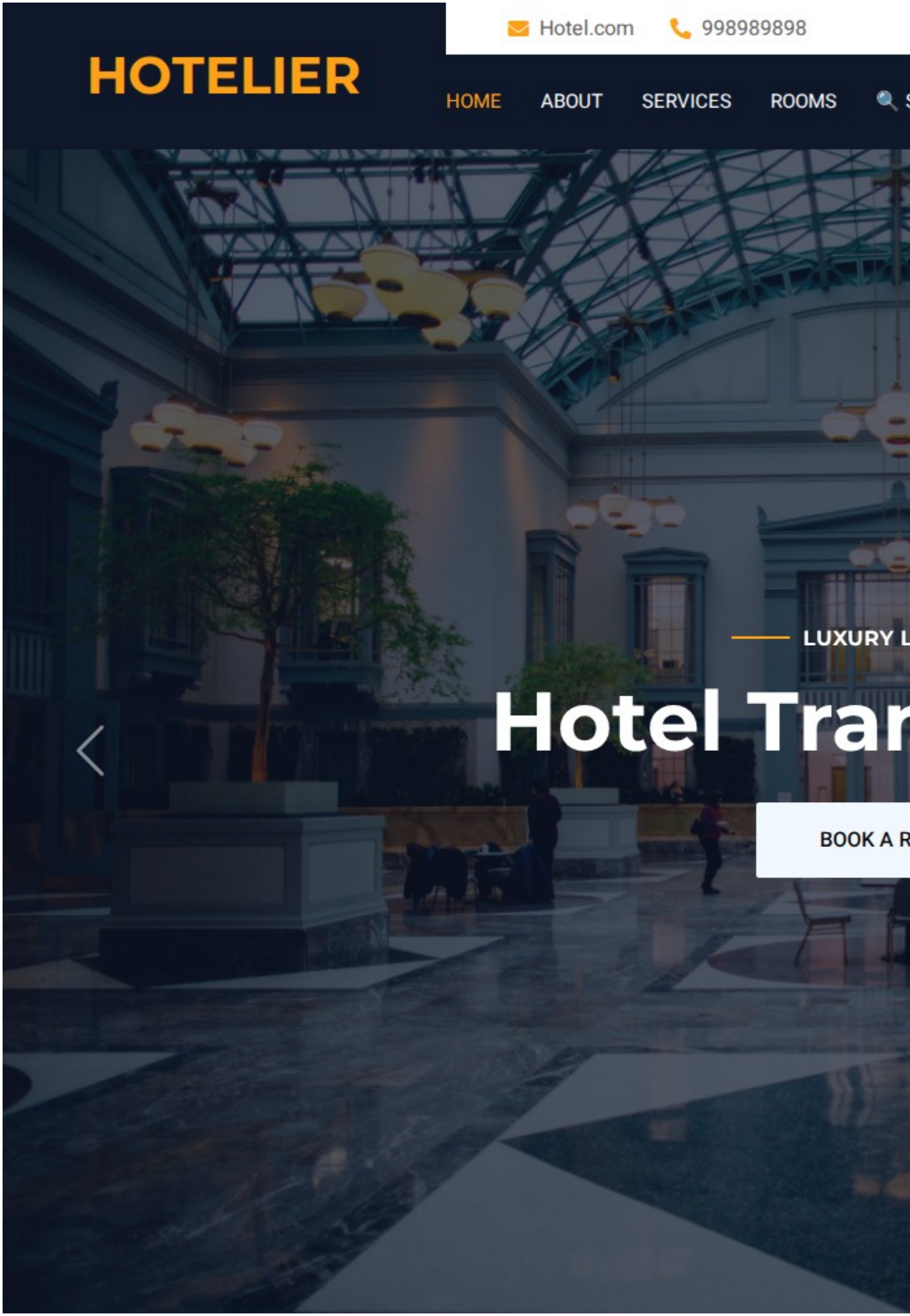

Hotel.com
998989898
HOTELIER
HOME
ABOUT
SERVICES
ROOMS
LUXURY L
Hotel Tran
BOOK A R

Fig 7.1: Home page with room type category cards, navigation bar, and chatbot widget

7.2 User Sign-In Page

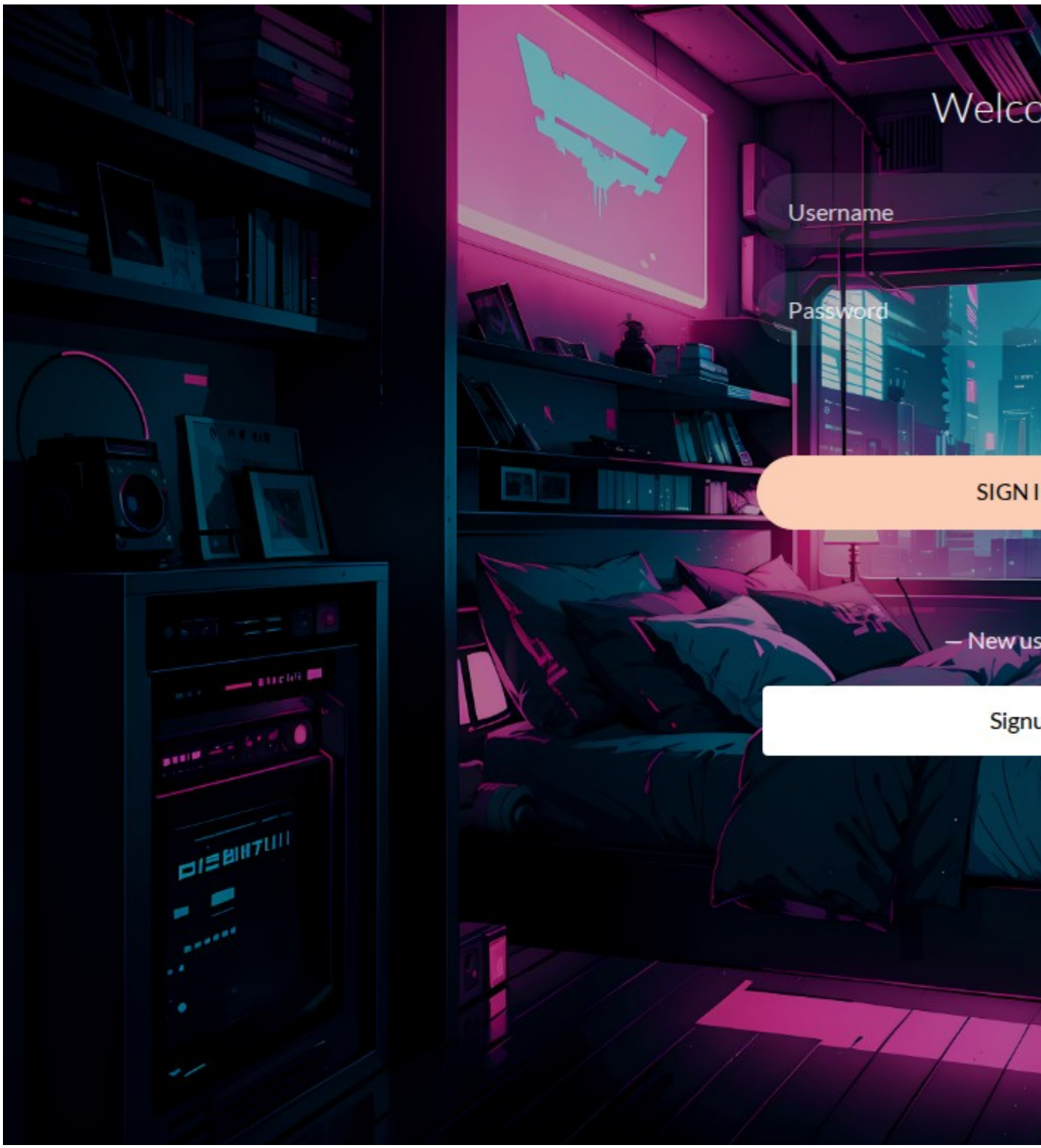
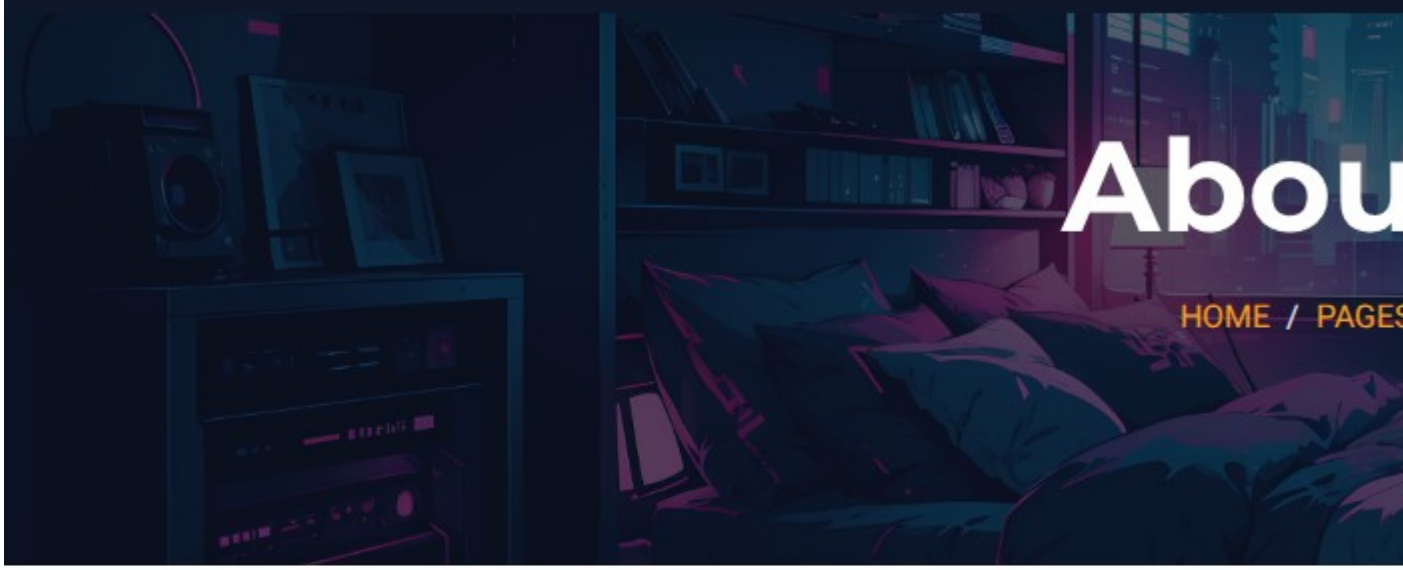


Fig 7.2: Sign-in form with username, password fields, and "Forgot Password" link

7.3 About Page



ABOUT US

Welcome to HOTELIER

hi erat elitr rebum at clita. Diam dolor diam ipsum sit. Aliqu diam amet diam et eos. Clita erat ipsum et lorem et sit, sed stet lorem sit clita duo justo magna dolore erat amet

858
Rooms

858
Staffs

858
Clients

EXPLORE MORE

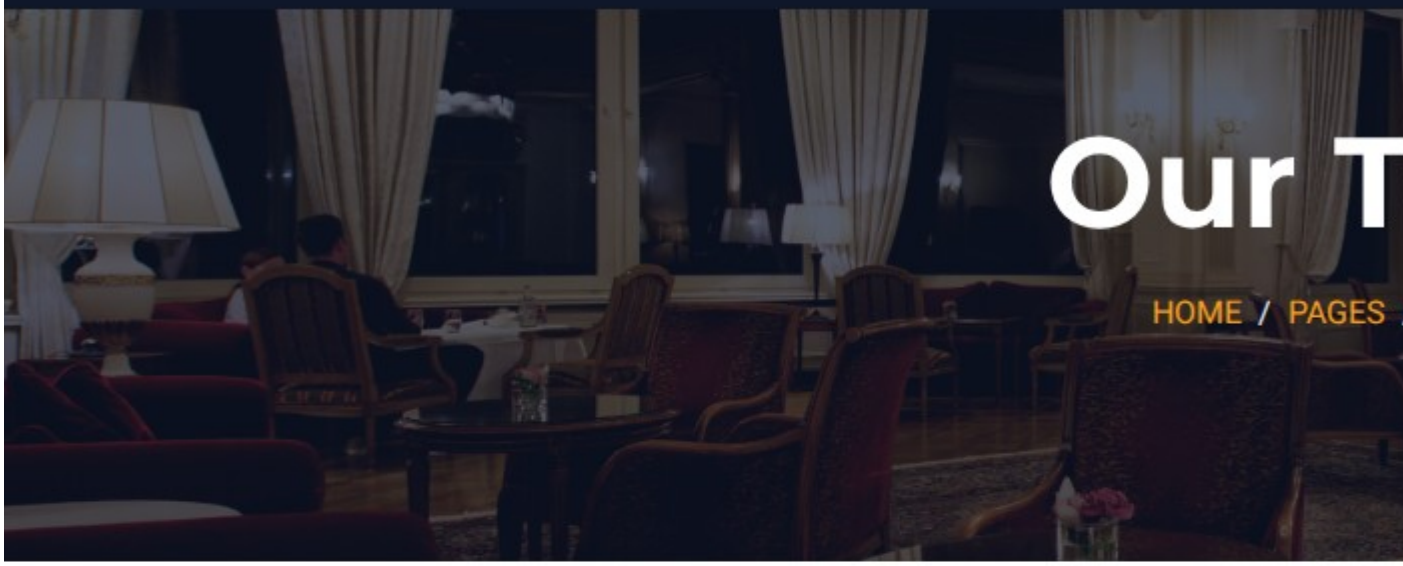
Fig 7.3: About page with hotel description and team information

7.4 Our Team Page

Hotel.com 998989898

HOTELIER

HOME ABOUT SERVICES ROOMS OTH



Our T

HOME / PAGES

OUR TE

Explore Ou

J

John Doe
Manager

J

Jane Smith
Receptionist

Fig 7.4: Team page with staff cards showing photos, names, and designations

7.5 Room Categories / Listing Page

Roo

OUR RO

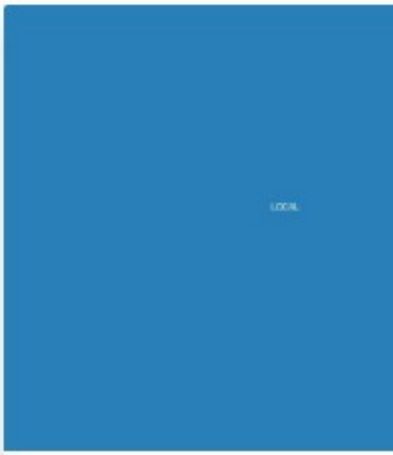
Explore Ou

luxury

1 Bed | 1 Bath | Wifi

swrsdsd

VIEW DETAIL



local

1 Bed | 1 Bath

fef

VIEW DETAIL

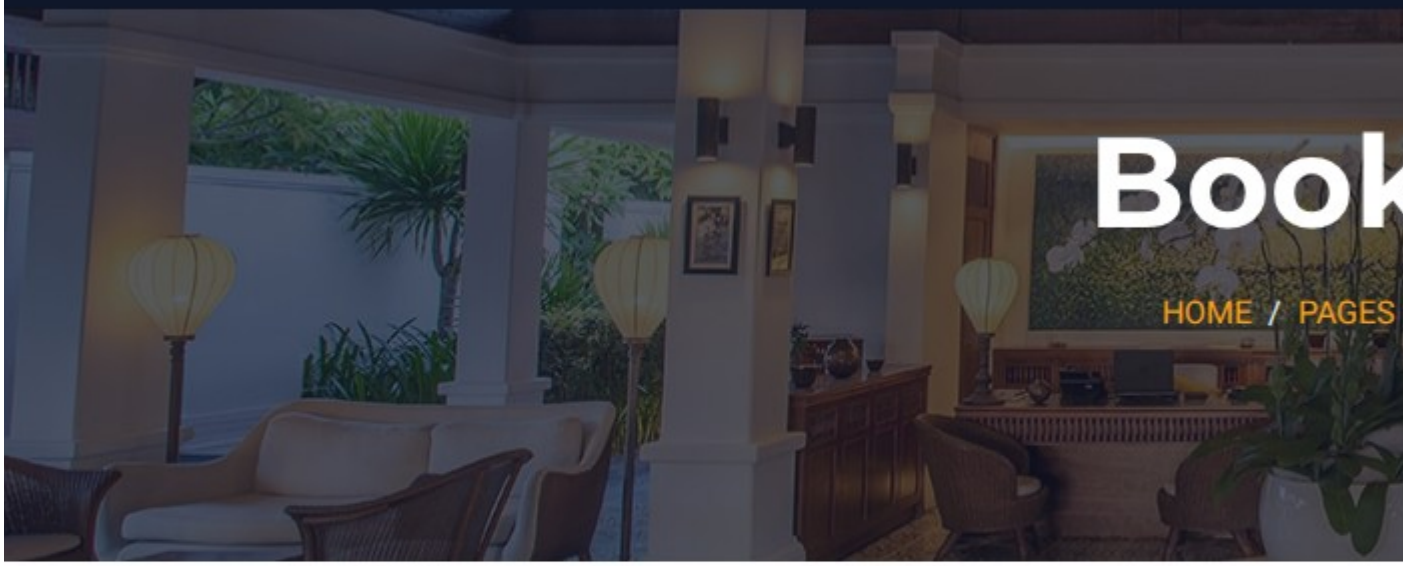
Fig 7.5: Room listing with images, prices, descriptions, and "Book Now" buttons

7.6 Booking Form Page

Hotel.com 998989898

HOTELIER

HOME ABOUT SERVICES ROOMS OTH



Book

HOME / PAGES

ROOM BO

Book A LUX

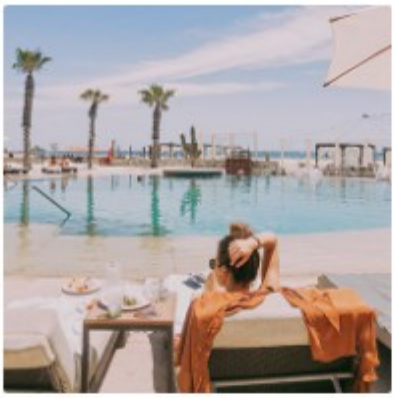


Fig 7.6: Booking form with customer details, dates, guest count, and calculated total

7.7 Admin Dashboard

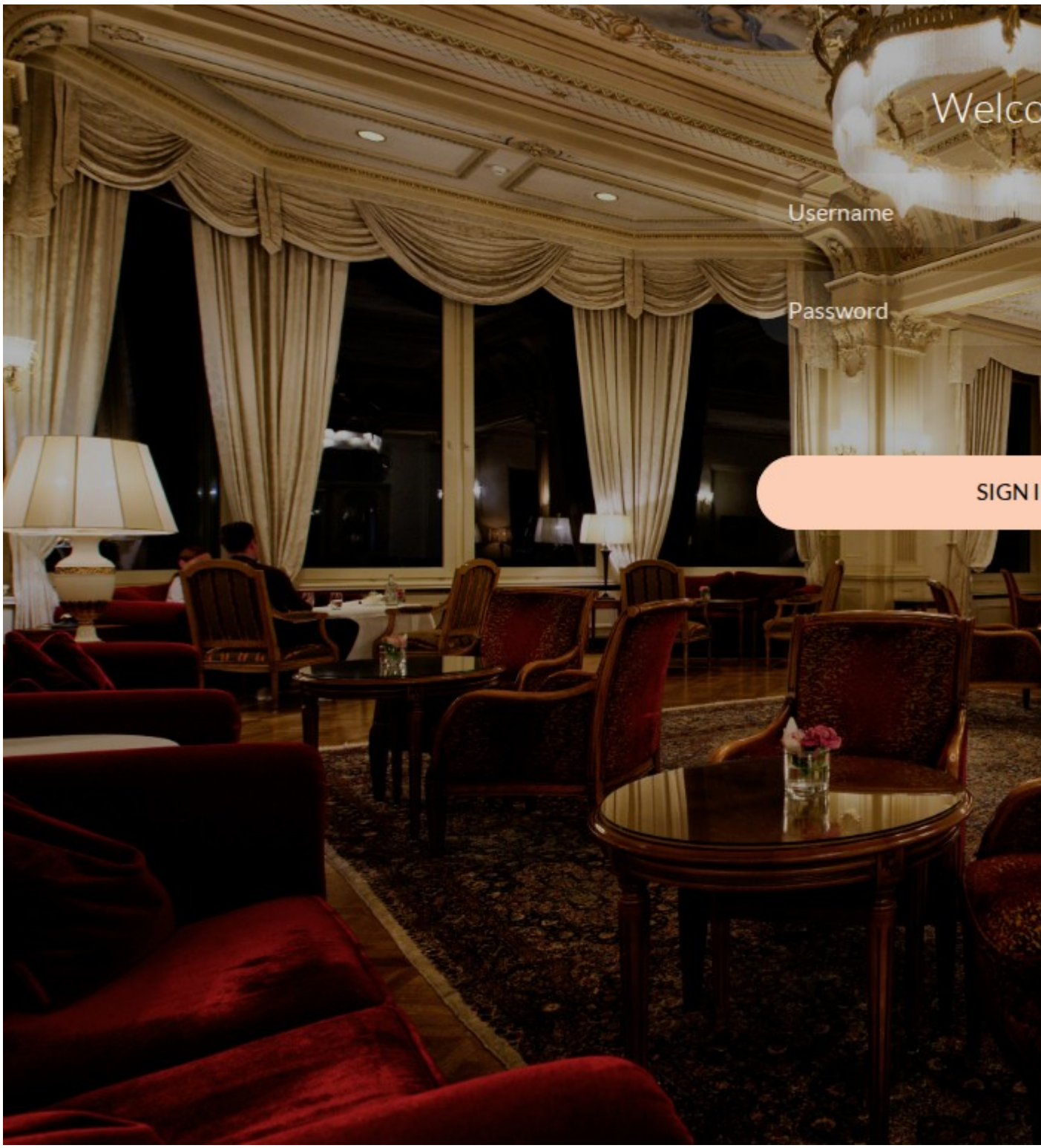


Fig 7.7: Admin dashboard with sidebar navigation and management overview

7.8 Admin – Room Type Management

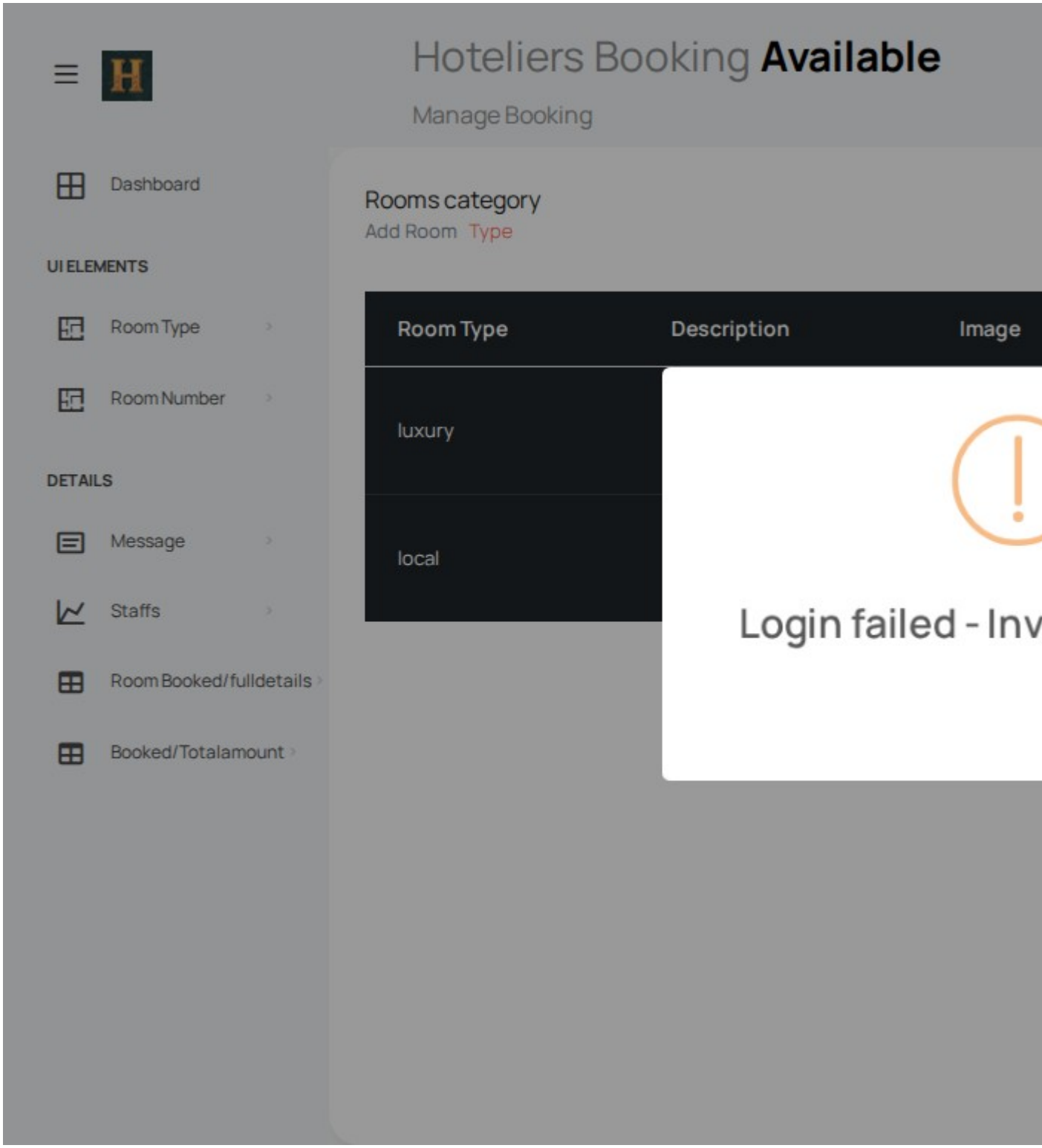


Fig 7.8: Admin room types page with add, edit, and delete actions

7.9 Admin – Room Management

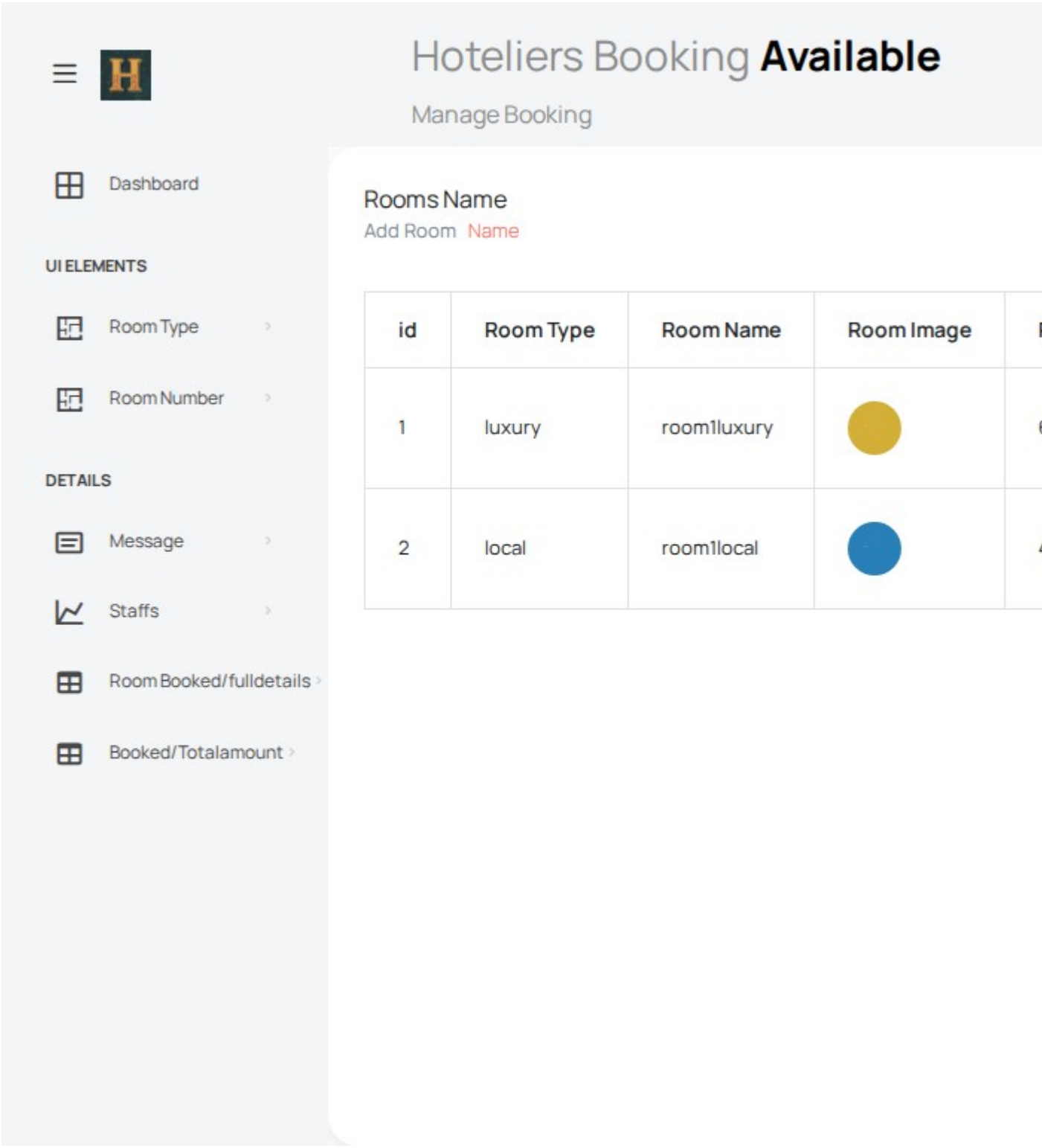


Fig 7.9: Admin room management with type, price, thumbnails, and actions

7.10 Admin – Staff Management

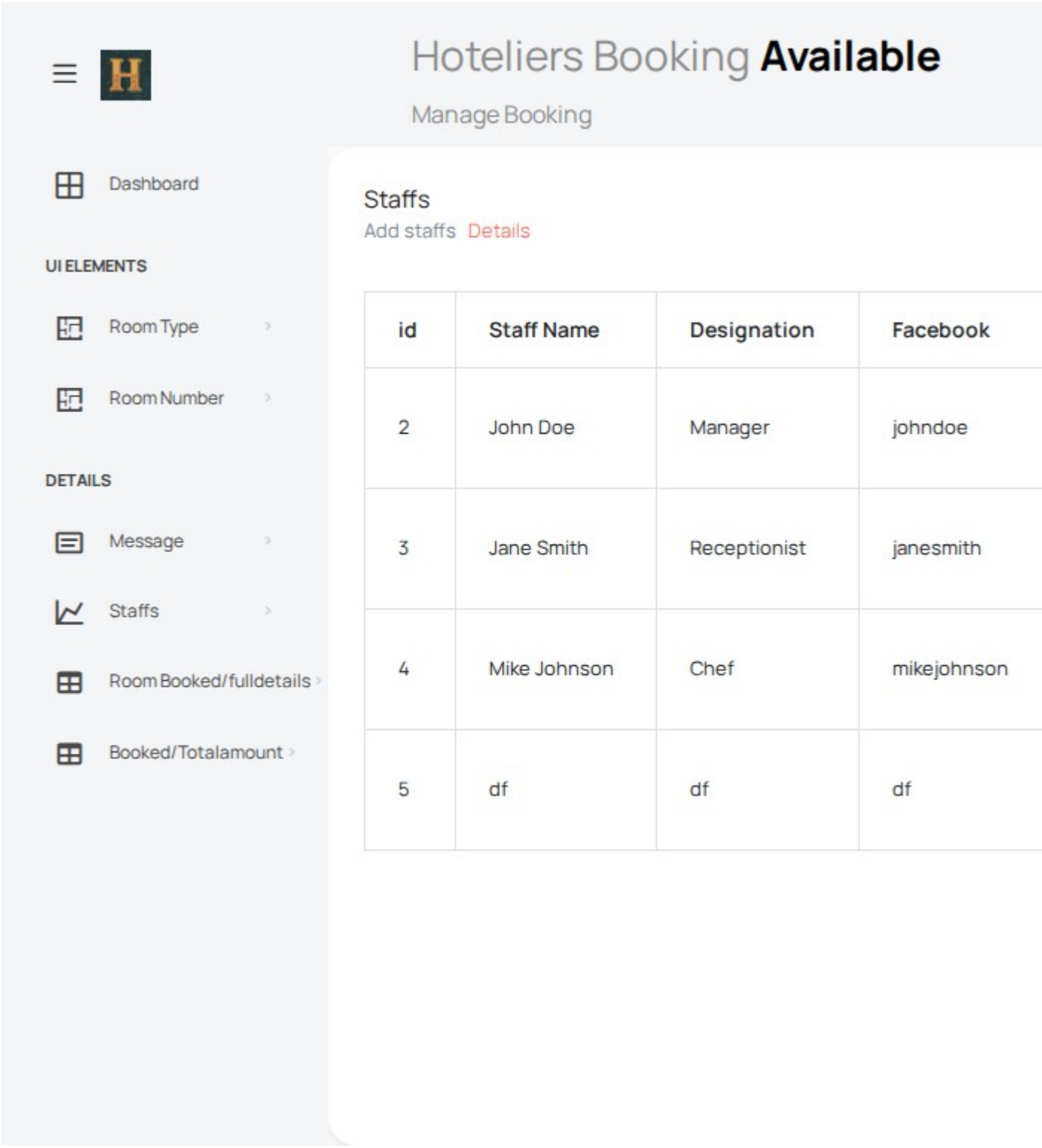


Fig 7.10: Admin staff page with names, designations, and edit/delete options

7.11 Admin – Booking Records

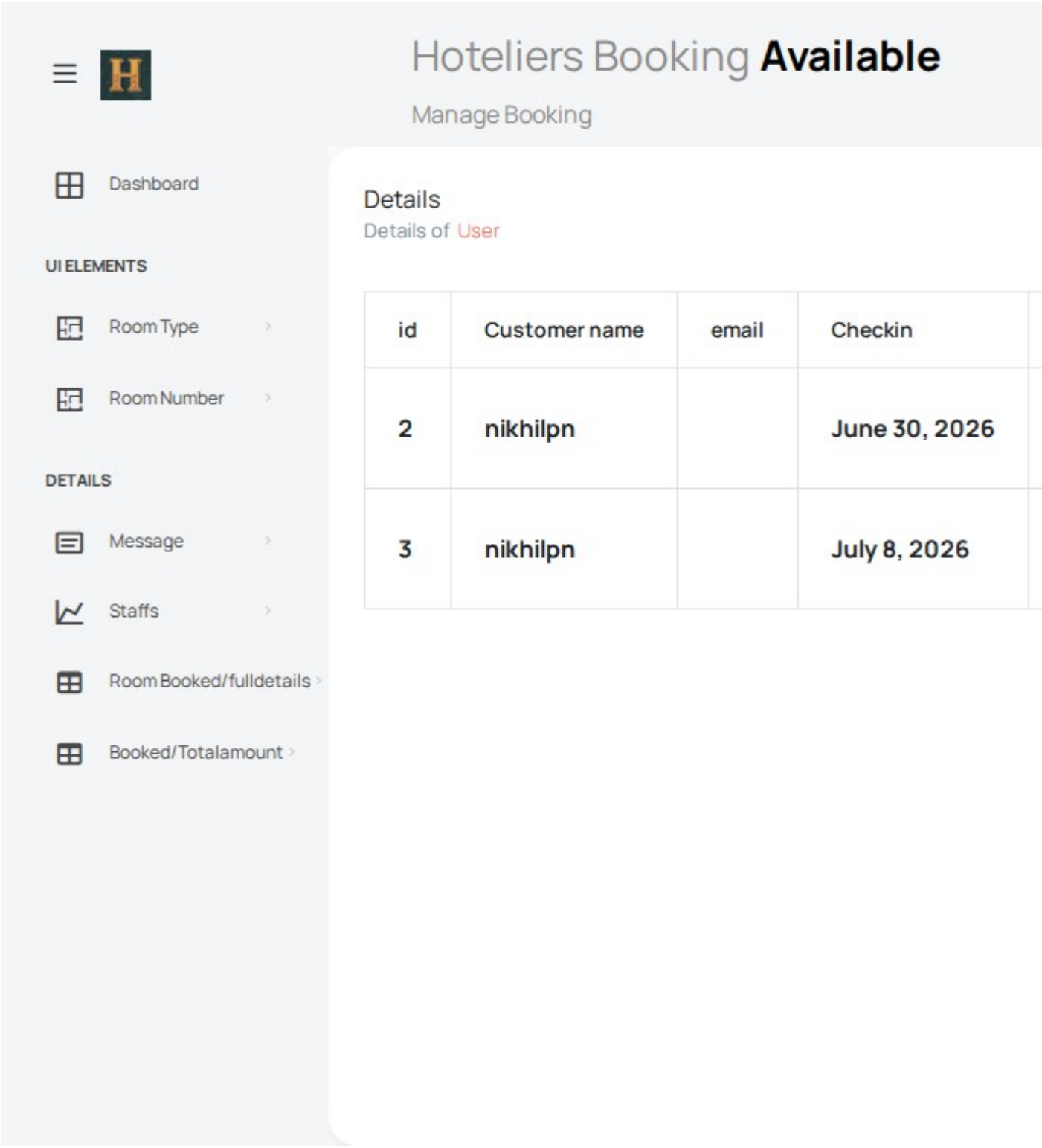


Fig 7.11: Admin bookings page with customer, room, dates, and price details

8. Implementation Plan

Phase	Activities	Duration
1. Requirements Analysis	Study existing system, define SRS, scope	1 week
2. System Design	ER diagram, normalization, module design, UI mockups	1 week
3. Backend Development	Django models, admin views, authentication, APIs	2 weeks
4. Frontend Development	HTML templates, Bootstrap, AJAX, chatbot UI	2 weeks
5. Integration	Razorpay, email, PDF, chatbot engine	1 week
6. Testing	Unit, integration, system, UAT, bug fixes	1 week
7. Documentation	Report writing, screenshots, final review	1 week

9. Limitations of the Project

1. Requires modern OS with Python 3.10+, Django 6.0, PostgreSQL 16.
2. Rule-based chatbot – no NLP or ML-based response generation.
3. Plain-text passwords in Registerdb (not hashed).
4. Single-hotel design – no multi-branch support.
5. Fixed room prices – no dynamic/seasonal pricing.
6. No automated refund processing for cancellations.
7. No native mobile app (responsive web only).
8. English-only interface – no i18n.

10. Future Application of the Project

1. **Dynamic Pricing:** Algorithm adjusting prices by demand and season.
2. **Loyalty Program:** Rewards with points and discount coupons.
3. **Multi-Hotel Support:** Centralized platform for multiple branches.
4. **AI/NLP Chatbot:** Upgrade to Hugging Face Transformers or OpenAI API.
5. **OTA Integration:** Sync with Booking.com, MakeMyTrip APIs.
6. **Advanced Reports:** Interactive charts for occupancy and revenue.
7. **Customer Reviews:** Ratings and reviews for rooms.
8. **SMS Notifications:** Via Twilio or MSG91 API.
9. **Cloud Deployment:** AWS, GCP, or DigitalOcean.
10. **Mobile App:** React Native or Flutter companion apps.
11. **Multi-Language:** Django i18n for global customers.
12. **Hashed Passwords:** Django User model with Argon2.
13. **Amenity Tagging:** Filter rooms by Wi-Fi, AC, TV, etc.

11. Conclusion

The **Hotel Room Booking Website with Assistant Chatbot** has been successfully designed, developed, and tested as part of the BCSP-064 project for the BCA programme at IGNOU. The system demonstrates practical full-stack web development using Django 6.0, Python, PostgreSQL, Bootstrap 5.3, JavaScript, and AJAX.

All objectives were achieved: automated room and booking management with overlap detection, Razorpay payment integration, assistant chatbot, email notifications, PDF report generation, AJAX search, and responsive design. The testing phase confirmed all functional and non-functional requirements are met. The modular architecture ensures maintainability and provides a foundation for future enhancements such as multi-hotel support, AI chatbot, and mobile applications.

This project has provided invaluable hands-on experience in web development, database design, payment integration, API development, and software engineering, and has significant real-world potential for small and medium hotels.

12. Bibliography

12.1 Websites and Online Documentation

1. Django Official Documentation – *https://docs.djangoproject.com/*
2. Python Official Documentation – *https://www.python.org/doc/*
3. PostgreSQL Documentation – *https://www.postgresql.org/docs/*
4. Razorpay Django Integration – *https://razorpay.com/docs/payments/server-integration/django/*
5. Bootstrap 5 Documentation – *https://getbootstrap.com/docs/*
6. MDN Web Docs – JavaScript – *https://developer.mozilla.org/en-US/docs/Web/JavaScript*
7. xhtml2pdf – *https://xhtml2pdf.readthedocs.io/*

12.2 Reference Books

1. Adrian Holovaty, Jacob Kaplan-Moss – *The Definitive Guide to Django*, 2nd Ed., Apress.
2. William S. Vincent – *Django for Beginners*, WelcomeToCode.
3. Eric Matthes – *Python Crash Course*, 3rd Ed., No Starch Press.
4. Antonio Melé – *Django 4 by Example*, 4th Ed., Packt Publishing.
5. Regina Obe, Leo Hsu – *PostgreSQL: Up and Running*, 3rd Ed., O'Reilly.
6. Abraham Silberschatz – *Database System Concepts*, 7th Ed., McGraw-Hill.

12.3 IGNOU Study Materials

1. BCS-051: Introduction to Software Engineering
2. BCS-053: Web Programming

3. BCSL-057: Web Programming Lab
4. BCSP-064: BCA Project Guidelines